

05.12.2024г.

Метапрограмиране

Лекция 17

`type(b'abcd')` → създава обект от тип `bytes`, като всеки елемент от байтовия обект е число от 0 до 255

Като използваме `b'...'` можем да въвеждаме само ASCII символи.
Иначе грешка.

Isinstance

Проверява дали даден обект принадлежи към определен тип или клас, или наследява от него.

`isinstance(3, object)` # True

`isinstance(3, int)` # True

`isinstance(3, type)` # False

↓
int, но не е клас

`isinstance(int, type)` # True

Issubclass

Проверява дали първият аргумент е подклас на втория.

`issubclass(3, int)` # Грешка, защото 3 не е клас

`issubclass(object, int)` # False

`issubclass(int, object)` # True

`issubclass(int, type)` # False

`object` е базовият клас за всички класове

`issubclass(t, t) == True`

Всеки клас е подклас на себе си,

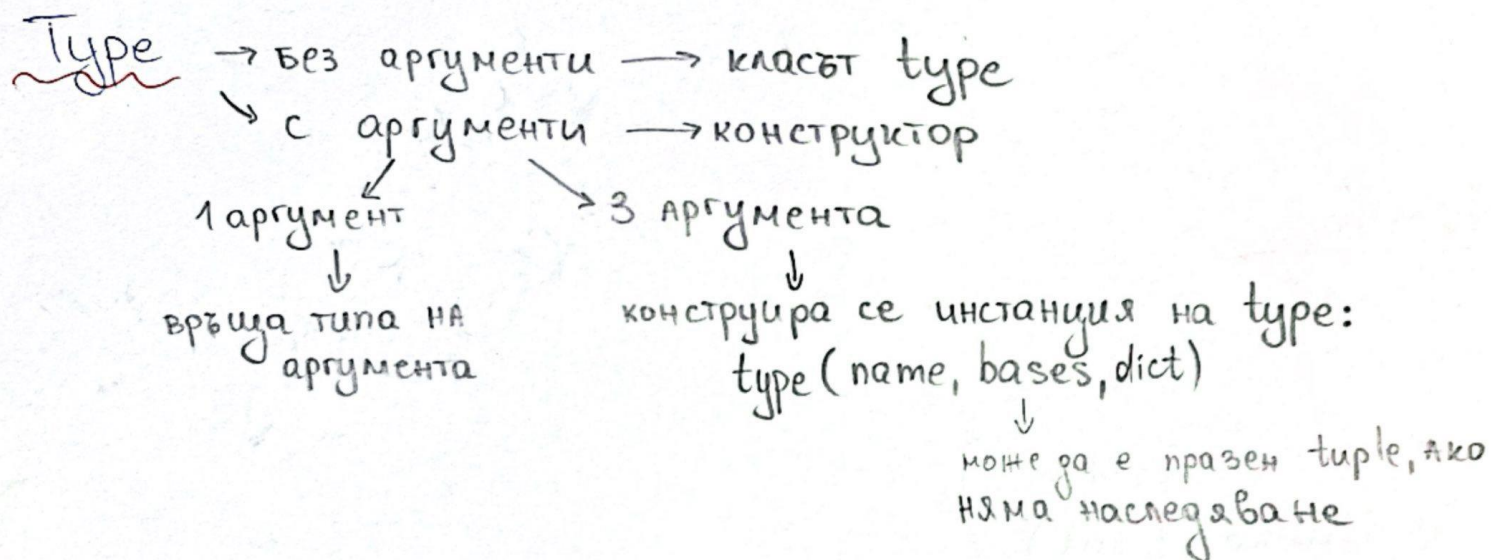
иначе обхожда `--bases--` и търси съвпадение

`isinstance(x, t) == issubclass(x.__class__, t)`

Метаклас

класът на класовете

type е метаклас на int, str, object ...



Метапрограмиране

програми, които пишат програми
техника, при която програми третират други програми като данни

macros

или if-else условия

или рекурсия

lisp

'(1 2 3) - списък с числа

(+ 1 2) - 1+2

(foo x) - foo(x)

'(+ 1 2) - списък с функцията + и 1 и 2

Можем да си напишем наш Епит с метапрограмиране
или използваме готовия като го импортируем.

Абстрактни класове / методи → поддържа полиморфизма
свързват се с помощта на модул abc

Абстрактен клас → клас, който служи като шаблон за други класове

Абстрактен метод → имплементира се от всеки подклас
→ само се декларира

dis → използва се за дизасемблиране на Python байткод.

↳ с него можем да видим как интерпретаторът превежда кода в инструкции, които изпълнява